```
object ConsumerCommittableSource {
  ...
  val drainingControl: DrainingControl[_] =
    Consumer
      .committableSource(                                              ❶
        consumerSettings,
        Subscriptions.topics("test2"))
      .map { msg: CommittableMessage[String, String] =>                ❶
        println(
          s"${msg.record.key} => ${msg.record.value.toUpperCase}")
        msg.committableOffset                                          ❷
      }
      .via(Committer.flow(committerSettings.withMaxBatch(100)))       ❸
      .toMat(Sink.seq)(DrainingControl.apply)                          ❹
      .run()
  ...
}
```

❶ CommittableSource that produces records of CommittableMessages. Reads from topic “test2”.

❷ Retrieves the offset from the record to pass it to Committer.flow

❸ Sets the commit every 100 records or 10 seconds

❹ Drains the current messages on the stream before shutdown

You can use `Committer.sink(settings: CommitterSettings)` instead of `Committer.flow`, as follows:

```
      Consumer.committableSource(...)
       .map( ... )
       .toMat(Committer.sink(committerSettings)(DrainingControl.apply))
       .run()
```

If the commit is the last thing you want to do, this is the best option. But part of the flow may be nice to have but not a must in your application, such as telemetry. In that case, you can include telemetry after the `committer.flow`.